

Checkpoint 2 — Aluno 1

Papel na arquitetura

O aluno 1 implementa o caminho dos bytes do PDF: preparar o arquivo, dividir em chunks, comprimir, enviar, persistir, baixar e verificar a integridade. O Peer armazena os chunks; o Super Peer mantém o índice e as localizações. O upload só termina depois de confirmar o armazenamento e receber o ACK do anúncio ao Super Peer.

Arquivos principais

- `peer.c`: entrada da CLI e serviço do Peer de armazenamento. O antigo `peer_service.c` foi incorporado a esse arquivo.
- `file_client.c`: coordena upload e download, inclusive os workers paralelos.
- `storage.c`: mantém manifests e chunks em pending e objects, confirma ou retoma uploads.
- `content.c`: hash de buffers, nome do arquivo, extensão `.pdf` e sincronização de diretórios.
- `compression.c`: adaptação à biblioteca LZ4.
- `local_control.c`: canal Unix local entre a CLI e o Peer ativo.
- `network.c`, `protocol.c` e `transfer_protocol.c`: TCP, framing/CRC32 e payloads C2 compartilhados.

Fluxo do upload

1. O comando `peer upload` chega ao Peer ativo pelo socket Unix local. O Peer usa seu NodeID já registrado no Super Peer.
2. O nome deve terminar em `.pdf`, sem análise semântica do formato. O ObjectID é SHA-256 dos bytes completos do arquivo.
3. O PDF é dividido em chunks de até 4 MiB. Cada chunk recebe hash SHA-256 do conteúdo original e compressão LZ4 individual.
4. O fluxo envia `STORE/BEGIN` e `STORE/COMMIT`; os workers usam `pread` e conexões próprias para enviar os `STORE/CHUNK` ao Peer de armazenamento.
5. O receptor valida índice, posição, tamanhos e hash, e grava os chunks em pending. No `COMMIT`, recompõe o conteúdo, verifica tamanho e ObjectID e publica em objects.
6. O Peer anuncia o documento ao Super Peer com `ANNOUNCE`. O sucesso exibido na CLI depende do ACK desse anúncio.

Fluxo do download

1. O Peer ativo consulta o Super Peer por nome ou ObjectID com `LOOKUP`.
2. A resposta contém descritores, hashes e localizações dos chunks. O Peer busca cada chunk diretamente de um Peer de armazenamento.
3. Se uma localização falha, o worker pode tentar a próxima. Os dados recebidos passam por validação de mensagem, descompressão LZ4 e conferência do hash do chunk.
4. Os bytes são escritos no offset correto de um arquivo temporário `.part`. Após todos os chunks, o SHA-256 completo deve coincidir com o ObjectID.
5. O resultado é sincronizado e publicado sem sobrescrever um destino existente. A CLI exibe `Download completed` e `SHA-256 verified`.

Por que essas escolhas?

- Chunks de 4 MiB: permitem transferir e validar partes independentemente, com paralelismo e retomada após interrupção.
- LZ4 por chunk: usa biblioteca pronta e deixa cada parte descompressível sem depender das demais.

- OpenSSL e zlib: fornecem SHA-256 e CRC32 já implementados; o projeto não reimplementa esses algoritmos.
- Workers com conexões próprias: evitam compartilhar estado de framing e permitem tentar outra localização por chunk.
- pending, objects e .part: separam dados incompletos de dados publicados; uma falha não deve parecer sucesso nem substituir um arquivo válido.
- Canal Unix local: a CLI usa o NodeID do Peer ativo; não cria uma identidade TCP anônima para upload ou download.

Demonstração e testes

Em terminais separados, com as portas livres:

```
make -B all
./bin/superpeer --port 55101 --name superpeer
./bin/peer serve 55102 127.0.0.1 55101
./bin/peer serve 55103 127.0.0.1 55101
./bin/peer upload arquivo.pdf 127.0.0.1 55102
./bin/peer --local-peer-port 55103 download arquivo.pdf recebido.pdf 127.0.0.1 55101
cmp arquivo.pdf recebido.pdf
```

Para verificar automaticamente: `make test-c2`; `bash script_testes_c2.sh`; `python3 tests/c2/integration.py --bin-dir bin`. Na verificação desta versão, o roteiro C2 terminou com 20/20 e a integração TCP com 46 verificações aprovadas.

Limites da entrega

A aceitação de PDF usa a extensão, não um parser de PDF. A transferência foi testada nos cenários disponíveis, mas metas globais de disponibilidade, replicação e consistência distribuída dependem de checkpoints posteriores. O índice do Super Peer é volátil; reiniciar somente o Super Peer não solicita automaticamente anúncios dos Peers que permaneceram ativos.